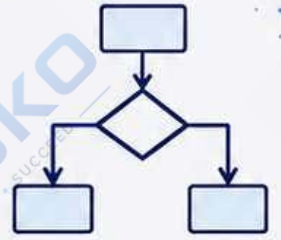


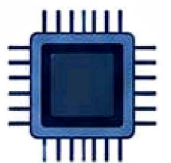
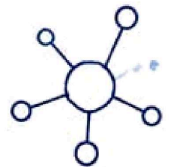
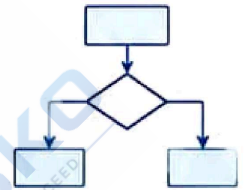
1010101010
0101010101
1010101010
0101010101



Notes



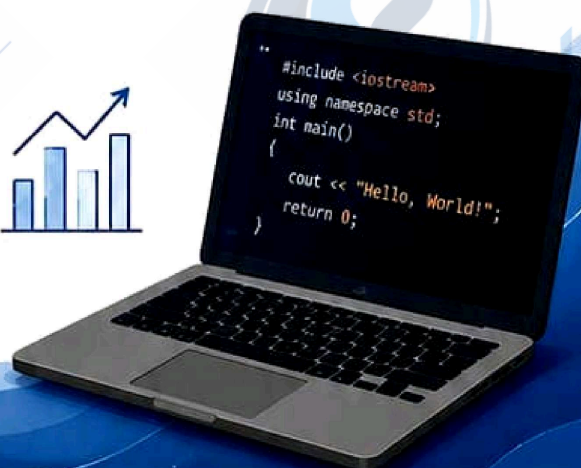
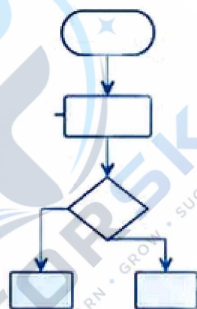
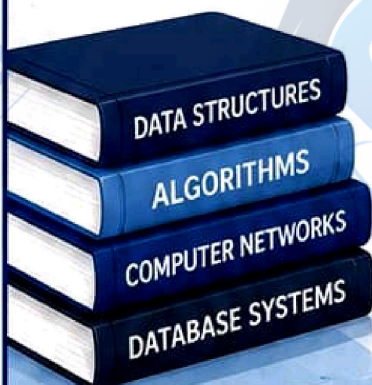
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



Chapter 2 -

7M

classmate

Date

Page

One - Sentence Question.

1. What is the purpose of `scanf()` in C?

→ `scanf()` is used for taking or reading any value from the keyboard by the user.

2. How does `getchar()` differ from `scanf()`?
Ans:- `getchar()` reads a single character from standard input.

• `scanf()` can read formatted input, including character strings, integers, etc.

3. What is the use of `puts()`?

Ans:- `puts()` function is used to display a group of characters or string which is already stored in a character array.

4. Define conditional operator in C?

Ans:- The conditional operator is used to add conditional code in our program. It is similar to `if-else` statement. It is also known as ternary operator.

5. What is the significance of the `break` statement?

Ans:- The significance of the `break` statement is to be used in a situation when we are not sure about the actual number of iterations for the loop or we want to terminate the loop based on some condition.

6. Explain the continue statement in a loop.

Ans. Continue statement skips the remainder body and immediately resets its condition before reiterating it.

7. What is nested if statement?

→ A nested if in C is an if statement that is the target of another if statement.

OR

8. Nested if statements mean an if statements within if statements.

8. How does switch statement work?

→ A switch statement checks the value of a variable or expression and runs the matching case block. If no match is found, it runs the default block (if present).

9. What is the syntax of a for loop?

→ `for (initialization; condition; update)`
`{ // code to be repeated. }`
eg-

`for (int i=0; i<5; i++)`

`{ printf("%d\n", i); }`

10. What does the goto statement do?

The goto statement in C also referred to as the unconditional jump statement can be used to jump from one point to another within a function.

Short Answer-

1.

Explain the difference between formatted and unformatted I/O in C.



Formatted I/O

1) These functions allow us to take input or display output in the user's desired format.

2) These functions support format specifiers.

3) These are used for storing more user-friendly

4) Here, we can use all data types.

5) eg- printf(), scanf(), sprintf() and sscanf()

Unformatted I/O.

1) These functions do not allow to take input or display output in the user's desired format.

2) These functions do not support format specifiers.

3) These functions are not user-friendly.

4) Here, we can only use character and string data types.

5) eg- getch(), puts(), getc(), getchar().

2. What is the purpose of `putchar()`? Provide an example.

→ The `putchar()` function is used to display a single character at a time by passing that character directly to it or by passing a ~~various~~ variable that has already stored a character.

eg- `#include <conio.h>`

`#include <stdio.h>`

`int main()`

{ `char ch;`

`printf("Enter any character;");`

`ch = getchar();`

// Reads the character

`putchar(ch);`

// Display character

`return 0;`

}

3. How does the while loop function work in C?

→ First initializes, then checks conditions, then executes the body, and updating can be inside the body.

4. Explain the usage of `?:` operator with an example?

→ Ternary operator is used to add a conditional code in the program.

It is also called as ternary operator because it takes 3 operands.

1. Condition -

2. Expression 1 - True

3. Expression 2 - False.

`int max(a < b ?`

5. What happens if a break statement is not used in a switch case?

→ If a break statement is not used in a switch case, the program will continue to execute the subsequent case statements until a break is encountered or the switch ends.

6. What are nested loops? Give an example.

→ Nested loops mean having one loop inside another loop. The inner loop runs completely every time the outer loop runs once.

eg-

```
#include <stdio.h>

int main ( )
{
    for (int i=1; i<=2; i++)
    {
        // Outer loop
        for (int j=1; j<=3; j++)
        {
            // Inner loop
            printf ("%d %d\n", i, j);
        }
    }

    return 0;
}
```

7. How does the do-while loop differ from the while loop?

→ In a while loop, the condition is checked before the loop body runs, so it may not run at all. In do-while loop, the loop body runs at least once because the condition is checked after the loop.

8. Why is goto generally discouraged in C?

→ Goto is generally discouraged in C because it breaks the normal flow of the program, making it difficult to understand and maintain. It can cause errors and make debugging harder, especially in large programs.

9. Write a simple C program using getch().

→ #include <stdio.h>

#include <conio.h>

int main()

{ printf("Hello world, press any key");

getch();

}

10. What is the difference between continue and break statements?

→ Break stops the loop completely.

Continue skips the current iteration and moves to next one.

Long Answer -

④

Explain printf() and scanf() functions in detail with examples.

→

printf() function is used in a C program to display any value like float, integer, characters, string, etc on the console screen. It is defined function that is already declared in the stdio.h. (Header file)

Syntax-

```
printf("Format Specifier", var1, var2, ... Var
```

eg-

```
#include <stdio.h> #include <stdio.h>
int main() int main()
{ {
    int a; int c;
    a = 10; c = 50;
    printf("%d", a); printf("%d", c);
    return 0; return 0;
}
```

Output -

Output

10

50

#include <stdio.h>

int main()

int a = 10;

printf("%d", a);

return 0;

}

scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any data type like integer, float, character string, and many more. This function is declared in `stdio.h` (header file) that why it is also pre-defined function.

Syntax -

```
scanf("Format Specifier", &var1, &var2...&varn);
```

Eg -

```
#include <stdio.h>
int main (C)
{
    int num;
    printf("Enter an integer number");
    scanf("%d", &num);
    printf("You have entered %d", num);
    return 0;
}
```

Output -

Enter an integer number: 26.
You have inter 26.

✓ 2. Differentiate between `getchar()`, `putchar()`, `gets()` and `puts()` with suitable examples.

	<u><code>getchar()</code></u>	<u><code>putchar()</code></u>	<u><code>gets()</code></u>	<u><code>puts()</code></u>
Defination - on	It reads a single character from the keyboard. It does not require arguments and returns the character read.	It displays a single character on the screen. It takes one character as an argument.	To read a whole line of text from the user until a new line is encountered. It stores the input in a character array.	To display a string on the screen. It automatically adds a new line character at the end.
Syntax	char variable = <code>getchar()</code> ;	<code>putchar(variable)</code> ;	<code>gets(string_variable)</code> ;	<code>puts(string_variable)</code> ;
eg-	<pre>#include <stdio.h> int main() { char ch; printf("Enter"); ch = getchar(); printf("%c", ch); return 0; }</pre>	<pre>#include <stdio.h> int main() { char ch = 'A'; printf("Character is: "); putchar(ch); return 0; }</pre>	<pre>#include <stdio.h> int main() { char n[50]; printf("name: "); gets(n); printf("Hello, %s", n); return 0; }</pre>	<pre>#include <stdio.h> int main() { char m[] = "Welcome"; puts(m); return 0; }</pre>

3. How does the w

2. Explain all branches statements in C with syntax and examples.

→ Branching statements allow the program to make decisions and change the flow of execution based on certain conditions.

The main branching statements in C are-

1. if Statement

The if statement checks a condition, and if it is true, it executes the block of code inside it. If condition is false, it skips the block.

Syntax -

```
if (condition)
{ // code to execute if condition is true }
```

Example -

```
#include <stdio.h>

int main()
{
    int age = 20;
    if (age >= 18)
    {
        printf("You are eligible to vote.");
    }
    return 0;
}
```

2. if-else Statement

The if-else statement provides two paths: one if the condition is true and another if the condition is false.

Syntax -

```
if (condition)
{ // Code if true }
else { // code if false }
```

Example -

```
#include <stdio.h>
int main ( )
{ int num = 5;

if (num % 2 == 0)
{ printf ("Even Number"); }
else { printf ("Odd number"); }
return 0;
}
```

Output -

Odd Number.

3. else if Ladder-

Used when there are multiple conditions to test. If checks conditions one by one from top to bottom and executes the block of the first true condition.

Syntax -

```
if (condition 1)
{ //Block 1 }

else if (condition 2)
{ //Block 2 }

else { //Default Block }
```

Example-

```
#include <stdio.h>

int main()
{
    int marks = 72;
    if (marks >= 90)
    { printf("Grade A"); }
    else if (marks >= 80)
    { printf("Grade B"); }
    else { printf("Grade C"); }
    return 0;
}
```

```
#include <stdio.h>

int main()
{
    int num;
    printf("Enter your percentage");
    scanf("%d", &num);
    if (num >= 90)
    { printf("Grade A"); }
}
```

4. Switch Statement.

The switch statement is used to choose one block of code from many based on the value of a variable or expression. It is useful when you have to compare a single variable with multiple constant values.

Syntax -

```
switch (expression)
{
    case value 1:
        // Code
        break;
    case value 2:
        // Code
        break;
    ....
    default:
        // Default code
}
```

Example -

```
#include <stdio.h>
int main ( )
{
    int day = 2;
    switch (day)
    {
        case 1:
            printf("Monday"); break;
        case 2:
            printf("Tuesday"); break;
        default:
            printf("Invalid day");
    }
    return 0; }
```

5. Explain the different types of loops in C with their syntax and examples.

→ A loop in C is used to execute a block of code repeatedly as long as a given condition is true. Loops help in reducing code repetition and writing efficient programs.

There are three main types of loops in C.

1. for Loop

When the number of repetitions is ~~unk~~ known in advance.

Syntax-

```
for (Initialization; condition; increment/decrement)
{
    // code to execute repeatedly
}
```

eg-

```
#include <stdio.h>

int main ()
{
    for (int i=1; i<=5; i++)
    {
        printf("Count: %d \n", i);
    }
    return 0;
}
```

2. while Loop.

When the number of repetitions is not known in advance. The loop continues as long as the condition is true.

Syntax (

while (condition)

{

// code to execute }

- The condition is checked before the first execution.
- If the condition is false initially, the loop body does not execute at all.

eg-

```
#include <stdio.h>
int main ()
{
    int i = 1;
    while (i <= 5)
    {
        printf("value: %d\n", i);
        i++;
    }
    return 0;
}
```

3. do-while Loop.

When you want the loop to run at least once, even if the condition is false at the beginning.

Syntax-

```
do {  
    // code to execute  
} while (condition);
```

- The loop body is executed first, and then condition is checked.
- If condition is true, then the loop continues.

Eg-

```
#include <stdio.h>  
  
int main (C)  
{  
    int i = 1;  
    do {  
        printf ("Number: %d \n", i);  
        i++;  
    } while (i <= 5);  
    return 0;  
}
```

6. How does the conditional operator work? Compare it with the if-else statement.
→ The conditional operator is also called the ternary operator in C. It is used to make a decision between two expressions based on a condition - similar to a compact if-else statement.

It is called "ternary" because it takes three operands-

1. A condition
2. An expression to execute if the condition is true.
3. An expression to execute if the condition is false.

Syntax-

condition ? expression1 : expression2;

- If condition is true, expression1 is executed
- If condition is false, expression2 is executed

eg-

```
#include <stdio.h>
int main() {
    int a=5, b=10;
    int max = (a > b) ? a : b;
    printf("Maximum is %d", max);
    return 0;
}
```

Feature	Conditional Operator (?:)	if-else Statement
Code Style	Short and compact (1 line)	Can be multiline and more verbose
Use Case	Best for assigning values	Best for running multiple statements
Readability	Less readable for complex logic	Easier to understand with complex logic.
Return Value	Return value	Does not return a value directly
Performance	Slightly faster for simple tasks	Comparable but depends on use
Example Usage	<code>int max = (a > b) ? a : b;</code>	<pre> if (a > b) max = a; else max = b; </pre>

7.

What are break and continue statements?
Explain with examples.

1. Break Statement-

The break statement is used to exit a loop or switch statement immediately regardless of whether the loop condition is true or false.

It is used to stop a loop when a certain condition is met.
and to exit a switch case.

Syntax-

break;

Eg-

```
#include <stdio.h>
int main ( )
{
    for (int i = 1; i <= 10; i++)
    {
        if (i == 5)
        {
            break;
        }
        printf ("%d \n", i);
    }
    return 0;
}
```

2. Continue Statement:

The continue statement is used to skip the current iteration of a loop and continue with the next one.

It is used to skip certain values or steps during looping. And to avoid running part of the loop when a condition is met.

Syntax-

Continue;

Eg-

```
#include <stdio.h>
int main ( )
{
    for (int i=1; i <= 5; i++)
    {
        if (i == 3) {
            continue;
        }
        printf("%d\n", i);
    }
    return 0;
}
```

Feature	break	continue
Function	Exits the loop/switch	Skips current iteration, continues loop
Effect	Terminates loop execution	Jumps to the next iteration
Use Case	When you want to stop entirely	When you want to skip one loop cycle.

- ✓ 8. Write a C program that demonstrate the use of nested loops.



Program to print star triangle-

```
#include <stdio.h>
int main()
{
    int i, j;

    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf(" *");
        }
        printf("\n");
    }

    return 0;
}
```

Explanation-

- The outer loop (for (i = 1; i <= 5; i++)) controls the number of rows.
- The inner loop (for (j = 1; j <= i; j++)) prints stars in each row.
- As i increases, the number of stars increases forming a triangle.

q. Discuss the advantages and disadvantages of using goto statements.

→ For The goto statement allows jumping to another part of the program using a label. It is a type of unconditional jump.

Syntax -

```
goto label;
```

```
// some code
```

```
label:
```

```
// code to jump to
```

Advantages -

1. Simplifies complex conditions -

In deeply nested loops or conditions, goto can be used to exit multiple levels of loops quickly.

2. Useful in error handling -

Many low-level programs use goto to jump to cleanup code when an error occurs.

3. Improves readability in some rare cases -

For very short programs, goto can sometimes make the code look less cluttered than multiple if or break statements.

4. Efficient jump in large switch cases -

It can be used to transfer control directly to another case or a block when needed.

Disadvantages -

1. Leads to spaghetti code -

Excessive use of goto can make code hard to read, understand, and maintain.

2. Breaks structured programming principles.

goto ignores function and block boundaries making the code less modular.

3. Harder to debug and test -

Programs with many goto statements can be difficult to trace, especially in large codebases.

4. Better alternatives exist -

break, continue, functions, and loops usually provide cleaner control flow.

10. Compare and contrast formatted and unformatted input/output functions in C.

1. Formatted I/O Functions.

These functions allow the user to input or output data in a specified format, using format specifiers like %d, %f, %c, etc.

Common Functions.

- `printf()` - for formatted output
- `scanf()` - for formatted input.

eg - `int age;`
`scanf("%d", &age);`
`printf("Age is %d", age);`

2. Unformatted I/O Functions.

These functions handle single characters or strings without formatting control. They are simpler but less flexible.

Common Functions.

- `getchar()` - takes a single character input
- `putchar()` - outputs a single character.
- `gets()` - takes a string input
- `puts()` - outputs a string.

eg-

```
char ch;  
ch = getchar();  
putchar(ch);
```

Comparison.

Feature	Formatted I/O	Unformatted I/O
Control	High (with format specifier)	Low
Functions	printf(), scanf()	getchar(), putchar(), gets(), puts().
I/O Types	Supports multiple data types	Mostly characters / strings
Flexibility	More flexible	Less flexible
Ease of Use	Slightly complex	Simple to use